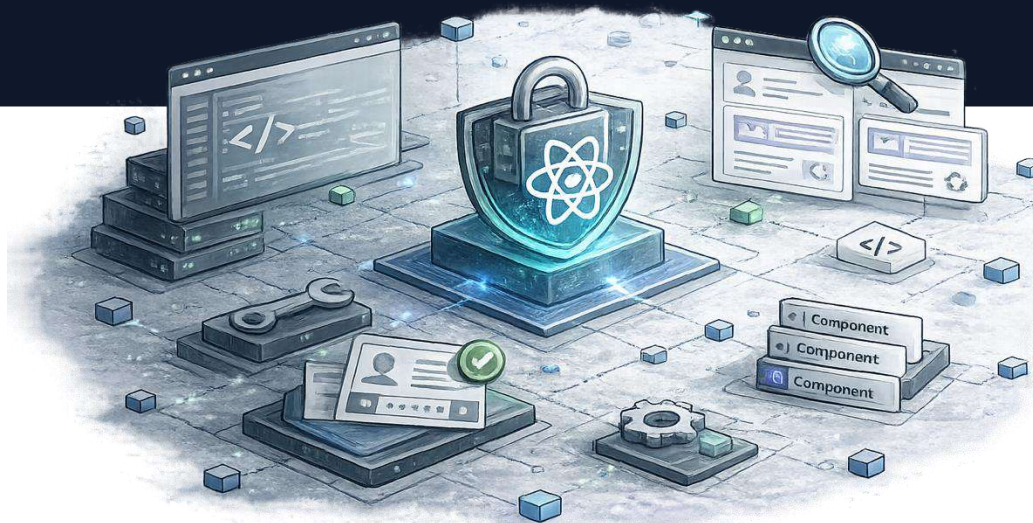




Progettazione di applicazioni web full stack

Sviluppo frontend con React



Prof. Devis Bianchini

Dip. di Università degli Studi di Brescia



- React.js è una libreria JavaScript/TypeScript per lo *«sviluppo front-end»* di applicazioni web
- Sviluppata nei laboratori Facebook 2011, utilizzata per la prima volta sull'applicazione Instagram nel 2012, attualmente utilizzata da aziende famose come AirBnB, Netflix, Paypal e Uber
- Rilasciata pubblicamente nel 2013 con licenza MIT (<https://opensource.org/license/mit>)



- React.js si ispira alla metodologia di sviluppo delle interfacce utente di tipo *SPA* («*Single Page Applications*»)
 - Un contenitore dentro il quale la pagina web evolve dinamicamente in funzione dei dati che arrivano dal back-end
 - Si contrappone all'approccio classico in cui il browser carica nuove pagine in seguito all'interazione con l'utente
 - Lo sviluppo di una pagina web avviene attraverso la scrittura di «*componenti*», che interagiscono con le API della libreria React.js che, a loro volta, manipolano il DOM per la creazione di elementi dell'interfaccia web

Concetto chiave: i componenti



- In React le app sono fatte di **«componenti»**
- I componenti sono oggetti complessi e dinamici, che ricevono input dall'esterno (interazioni utente, time-out, comandi dal back-end) e forgianno l'elemento grafico da restituire
- Un componente = un «pezzo» di UI con:
 - logica
 - aspetto (HTML o altri componenti annidati)
- I componenti possono essere:
 - piccoli (button)
 - grandi (intere pagine)

Un primo «assaggio»



- File `apps/ui/src/app/main.tsx`

StrictMode è un contenitore speciale che aiuta durante lo sviluppo

- Segnala pratiche sconsigliate
- Aiuta a scovare effetti collaterali
- Non aggiunge interfaccia visibile all'utente

```
import { StrictMode } from 'react';
import { BrowserRouter } from 'react-router-dom';
import * as ReactDOM from 'react-dom/client';
import App from './app/app';

const root = ReactDOM.createRoot(
  document.getElementById('root') as HTMLElement
);

root.render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

Un primo «assaggio»



- File `apps/ui/src/app/main.tsx`

- **BrowserRouter** gestisce la navigazione all'interno di una SPA (*Single Page Application*)
- È in grado di cambiare URL come `/`, `/about`, `/login` senza ricaricare l'intera pagina

```
import { StrictMode } from 'react';
import { BrowserRouter } from 'react-router-dom';
import * as ReactDOM from 'react-dom/client';
import App from './app/app';

const root = ReactDOM.createRoot(
  document.getElementById('root') as HTMLElement
);

root.render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

Un primo «assaggio»



- File `apps/ui/src/app/main.tsx`

- `ReactDOM` permette di usare le API del DOM per gestire gli elementi della pagina HTML
- La parte `/client` indica che si stanno utilizzando l'API moderna, quella introdotta da React 18

```
import { StrictMode } from 'react';
import { BrowserRouter } from 'react-router-dom';
import * as ReactDOM from 'react-dom/client';
import App from './app/app';

const root = ReactDOM.createRoot(
  document.getElementById('root') as HTMLElement
);

root.render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

Un primo «assaggio»



- File `apps/ui/src/app/main.tsx`

- Importa **App**, ovvero il componente principale dell'applicazione
- Solitamente, il contenitore principale dell'intera applicazione

```
import { StrictMode } from 'react';
import { BrowserRouter } from 'react-router-dom';
import * as ReactDOM from 'react-dom/client';
import App from './app/app';

const root = ReactDOM.createRoot(
  document.getElementById('root') as HTMLElement
);

root.render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

Un primo «assaggio»



- File `apps/ui/src/app/main.tsx`

- Cerca (nel file `index.html`, che è l'entry point dell'applicazione) un elemento con `id='root'`
- Certifica che l'elemento esiste ed è un elemento HTML (`as HTMLElement`)
- Crea una root React moderna (`ReactDOM.createRoot`), salvandola nella variabile `root`

```
import { StrictMode } from 'react';
import { BrowserRouter } from 'react-router-dom';
import * as ReactDOM from 'react-dom/client';
import App from './app/app';

const root = ReactDOM.createRoot(
  document.getElementById('root') as HTMLElement
);

root.render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

Un primo «assaggio»



- File `apps/ui/index.html`

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <title>Ui</title>
    <base href="/" />

    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <link rel="icon" type="image/x-icon" href="/favicon.ico" />
    <link rel="stylesheet" href="/src/styles.css" />
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

Un primo «assaggio»



- File `apps/ui/src/app/main.tsx`

Renderizza «qualcosa» dentro la root appena creata:

- `<StrictMode>` - contenitore esterno, attiva controlli durante lo sviluppo
- `<BrowserRouter>` - avvolge l'app per abilitare il routing (tutti i componenti dentro `App` potranno usare funzionalità di `react-router-dom`, come `Routes`, `Route`, `Link`, `useNavigate`)

```
import { StrictMode } from 'react';
import { BrowserRouter } from 'react-router-dom';
import * as ReactDOM from 'react-dom/client';
import App from './app/app';

const root = ReactDOM.createRoot(
  document.getElementById('root') as HTMLElement
);

root.render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

Un primo «assaggio»



- File `apps/ui/src/app/src/app/app.tsx`

- Importa il componente `NxWelcome`, per usarlo all'interno della pagina
- Il componente `NxWelcome` ha un parametro `title`

```
import NxWelcome from './nx-welcome';

import { Route, Routes, Link } from 'react-router-dom';

export function App() {
  return (
    <div>
      <NxWelcome title="@org/ui" />
    </div>
  );
}
```

Un primo «assaggio»



- File `apps/ui/src/app/src/app/nx-welcome.tsx`

- Esporta una funzione chiamata `NxWelcome`, rendendola utilizzabile in altri file (tramite `import { NxWelcome } ...`)
 - In React, si tratta di un ***componente funzionale***
- La funzione riceve un oggetto e da questo viene estratta subito la proprietà `title` (***destructuring***)
- La proprietà `title` deve essere di tipo `string` (TypeScript)

```
export function NxWelcome({ title }: { title: string }) {
  return (
    <>
      <style
        dangerouslySetInnerHTML={{
          __html: `
html {
  -webkit-text-size-adjust: 100%;
  font-family: ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont,
  'Segoe UI', Roboto, 'Helvetica Neue', Arial, 'Noto Sans', sans-serif,
  'Apple Color Emoji', 'Segoe UI Emoji', 'Segoe UI Symbol',
  'Noto Color Emoji';
  line-height: 1.5;
  tab-size: 4;
  scroll-behavior: smooth;
}
`
        }}
      />
    </>
  )
}
```

- In alternativa...

```
export function NxWelcome(prop: {title: string}) {
  const title = prop.title;
```

Un primo «assaggio»



- File `apps/ui/src/app/src/app/nx-welcome.tsx`

- Alla fine del componente viene eseguito un'ulteriore istruzione **`export default`**
- Specifica l'export principale del file (***default***)
- Non è obbligatorio, ma si usa di frequente, soprattutto quando un file contiene un solo componente

```
export default NxWelcome;
```

- Permette un **`import`** più snello (senza graffe)

```
import NxWelcome from './nx-welcome';  
  
import { Route, Routes, Link } from 'react-router-dom';  
  
export function App() {  
  return (  
    <div>  
      <NxWelcome title="@org/ui" />  
    </div>  
  );  
}
```

Un primo «assaggio»



- All'interno di un componente React possono essere visualizzate diverse tipologie di dati:
 - Dati in input (**prop**)
 - Valori relativi allo stato del componente
 - Valori di variabili interne
- Deve essere utilizzata la notazione con parentesi graffe { }

```
<div id="welcome">
  <h1>
    <span>
      Hello there
    </span>
    Welcome {title} 🙌
  </h1>
</div>
```

Un primo «assaggio»



- File `apps/ui/src/app/src/app/app.tsx`

Routes è la parte dinamica della pagina:

- Legge l'URL corrente della SPA
- Se questo è '/', sostituisce questa parte della SPA con l'HTML che si trova nella proprietà `element`
- Funziona perché App è stata inclusa nei tag `<BrowserRouter>`
`</BrowserRouter>`
- Tutto ciò che è al di fuori di **Routes** non cambia

```
import NxWelcome from './nx-welcome';

import { Route, Routes, Link } from 'react-router-dom';

export function App() {
  return (
    <div>
      <NxWelcome title="@org/ui" />
    </div>
  );
}
```

```
<Routes>
  <Route
    path="/"
    element={
      <div>
        This is the generated root route.{ ' '}
        <Link to="/page-2">Click here for page 2.</Link>
      </div>
    }
  />
```

Un primo «assaggio»



- File `apps/ui/src/app/src/app/app.tsx`

- L'output della funzione `App`, così come il valore dell'attributo `element`, contiene dell'HTML che viene renderizzato nella pagina ed eventualmente anche dei componenti annidati
- Per «scrivere» questo contenuto, viene utilizzata la *sintassi JSX* (*JavaScript XML*)

```
import NxWelcome from './nx-welcome';

import { Route, Routes, Link } from 'react-router-dom';

export function App() {
  return (
    <div>
      <NxWelcome title="@org/ui" />
    </div>
  );
}
```

```
<Routes>
  <Route
    path="/"
    element={
      <div>
        This is the generated root route.{ ' '}
        <Link to="/page-2">Click here for page 2.</Link>
      </div>
    }
  />
</Routes>
```



- **JSX** (*JavaScript XML*) = HTML dentro JavaScript
- JSX permette allo sviluppatore di scrivere facilmente tag HTML all'interno di codice JavaScript e di piazzarli all'interno del DOM senza l'uso di metodi quali **createElement()** e/o **appendChild()**
 - React ha comunque delle funzioni per creare elementi HTML
- Regole:
 - Componenti → notazione «*camel-case*» (es. **<MyComponent>**)
 - HTML → minuscolo (es. **<div>**)
 - Ogni tag di apertura deve sempre averne uno di chiusura

Rendering condizionale



- Utilizzato per mostrare contenuti in base a condizioni
- La UI cambia in base allo stato

```
let content;  
if (isLoggedIn) {  
  content = <AdminPanel />;  
} else {  
  content = <LoginForm />;  
}  
return (  
  <div>  
    {content}  
  </div>  
)
```



- Fa uso della funzione **map()** per generare elementi
- Ogni elemento ha bisogno di una chiave (**key**)

```
const products = [
  { title: 'Cabbage', isFruit: false, id: 1 },
  { title: 'Garlic', isFruit: false, id: 2 },
  { title: 'Apple', isFruit: true, id: 3 },
];

let App = function ShoppingList() {
  const listItems = products.map(product =>
    <li
      key={product.id}
      style={{
        color: product.isFruit ? 'magenta' : 'darkgreen'
      }}
    >
      {product.title}
    </li>
  );

  return (
    <ul>{listItems}</ul>
  );
}
```



- React gestisce eventi come:
 - Click, double click, etc.
 - Input
- La sintassi è simile a JavaScript, ma in JSX

```
function MyButton() {  
  function handleClick() {  
    alert('You clicked me!');  
  }  
  
  return (  
    <button onClick={handleClick}>  
      Click me  
    </button>  
  );  
}
```

Stato (state)



- Lo stato è interno al componente
- Serve per:
 - Aggiornare la UI
 - Reagire alle interazioni
- Quando cambia lo stato, React aggiorna la UI

```
import { useState } from 'react';

let App = function MyApp() {
  return (
    <div>
      <h1>Counters that update separately</h1>
      <MyButton />
      <MyButton />
    </div>
  );
}

function MyButton() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }

  return (
    <button onClick={handleClick}>
      Clicked {count} times
    </button>
  );
}
```



- I dati possono essere:
 - Passati dall'alto (proprietà)
 - Condivisi tra componenti

```
import { useState } from 'react';

let App = function MyApp() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }

  return (
    <div>
      <h1>Counters that update together</h1>
      <MyButton count={count} onClick={handleClick} />
      <MyButton count={count} onClick={handleClick} />
    </div>
  );
}

function MyButton({ count, onClick }) {
  return (
    <button onClick={onClick}>
      Clicked {count} times
    </button>
  );
}
```

Componenti e stile (I)



- I componenti possono essere dotati di un proprio stile

- Direttamente nel componente (*inline*)
- Veloce, ma poco leggibile

- Inserimento dell'attributo `style={}` nel tag; l'attributo accetta come valore un oggetto JavaScript
- Notazione «*camel-case*» per le proprietà CSS
 - `font-weight` → `fontWeight`
- I valori delle proprietà CSS devono essere tutti racchiusi tra singoli apici
- Le proprietà CSS vanno separate da virgole

```
<div style={{marginTop: '2.5rem'}}>
  <h1 style={{fontSize: '3rem',
    fontWeight: '500',
    letterSpacing: '-0.025em',
    lineHeight: '1'}}>
    <span style={{display: 'block',
      fontSize: '1.875rem',
      fontWeight: '300',
      lineHeight: '2.25rem',
      marginBottom: '0.5rem'}}>
      Hello there
    </span>
    Welcome {title} 🙌
  </h1>
</div>
```

Componenti e stile (II)



- I componenti possono essere dotati di un proprio stile

- Utilizzando il classico file CSS separato

- Nel componente si usa **className** invece di **class**

- Semplice e standard, ma potenziali conflitti di nome

```
import './nx-welcome-style.css';
```

```
<div className="welcome">
  <h1>
    <span>
      Hello there
    </span>
    Welcome {title} 🙌
  </h1>
</div>
```

```
.welcome {
  margin-top: 2.5rem;
}

.welcome h1 {
  font-size: 3rem;
  font-weight: 500;
  letter-spacing: -0.025em;
  line-height: 1;
}

.welcome span {
  display: block;
  font-size: 1.875rem;
  font-weight: 300;
  line-height: 2.25rem;
  margin-bottom: 0.5rem;
}
```

Componenti e stile (III)



- I componenti possono essere dotati di un proprio stile

- Utilizzando i moduli CSS
 - Solitamente, un componente è associato al proprio modulo
 - Molto usato in Nx
- Nessun conflitto di nome

```
import styles from './nx-welcome.module.css';
```

```
<div className={styles.welcome}>  
  <h1>  
    <span>  
      Hello there  
    </span>  
    Welcome {title} 🙌  
  </h1>  
</div>
```

```
.welcome {  
  margin-top: 2.5rem;  
}  
  
.welcome h1 {  
  font-size: 3rem;  
  font-weight: 500;  
  letter-spacing: -0.025em;  
  line-height: 1;  
}  
  
.welcome span {  
  display: block;  
  font-size: 1.875rem;  
  font-weight: 300;  
  line-height: 2.25rem;  
  margin-bottom: 0.5rem;  
}
```

Componenti e stile (IV)



- I fogli di stile (globali) possono anche essere linkati al file **index.html**

- Validi per tutti i componenti
- Nei componenti, si usa l'attributo **className**

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <title>Ui</title>
    <base href="/" />

    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <link rel="icon" type="image/x-icon" href="/favicon.ico" />
    <link rel="stylesheet" href="/src/styles.css" />
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

Componenti e stile (V)



- Un modo «non standard» e più pericoloso

- Usa un **React Fragment** (per raggruppare più elementi, senza utilizzare un `<div>`)
- L'attributo **`dangerouslySetInnerHTML`** serve per forzare React ad accettare l'inserimento di HTML non filtrato
- La proprietà `_html` indica obbligatoriamente il contenuto HTML da inserire
- Può essere pericoloso perché può essere soggetto a vulnerabilità **XSS** (cross-site scripting)

```
export function NxWelcome({ title }: { title: string }) {
  return (
    <>
      <style
        dangerouslySetInnerHTML={{
          __html: `
html {
  -webkit-text-size-adjust: 100%;
  font-family: ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont,
  'Segoe UI', Roboto, 'Helvetica Neue', Arial, 'Noto Sans', sans-serif,
  'Apple Color Emoji', 'Segoe UI Emoji', 'Segoe UI Symbol',
  'Noto Color Emoji';
  line-height: 1.5;
  tab-size: 4;
  scroll-behavior: smooth;
}
`
        }}
      />
    </>
  )
}
```



- Viene introdotto il concetto di **Virtual DOM**
- Al verificarsi di un evento, invece di manipolare il DOM del browser (come fanno altre librerie simili), React.js manipola un Virtual DOM, ovvero una copia esatta del DOM del browser che si trova in memoria centrale
- La manipolazione del Virtual DOM è più «leggera» di quella del DOM del browser
- Lavorando con il Virtual DOM, React.js sarà in grado di inviare al DOM del browser solo le modifiche strettamente necessarie, rendendo così più leggero il processo di rendering della pagina

Un esempio concreto di rendering



- Supponiamo che nel browser sia rappresentata una lista con 10 check-box e l'utente effettui il check del primo elemento
- La maggior parte dei framework javascript richiederebbero al DOM il rendering dell'intera lista (ovvero, 10 volte il lavoro che invece servirebbe, perché un solo elemento è cambiato mentre i restanti nove sono rimasti inalterati)
- In React.js, invece, l'evento di check attiva la manipolazione del Virtual DOM (operazione molto più agile perché non deve fare rendering su schermo)
- In seguito, React confronta il Virtual DOM appena manipolato con una copia dello stesso fatta precedentemente alla manipolazione (operazione chiamata «*diffing*»)
- Così facendo, individua solo gli oggetti che sono realmente cambiati: nel nostro caso, solo il primo check-box della lista; React.js comunica questa informazione al DOM del browser, il quale provvederà al rendering del solo check-box modificato



Per lo sviluppatore

- L'approccio a componenti, oltre che abilitare il riuso, permette di costruire interfacce complesse attraverso la composizione di semplici «mattoncini»
- Lo sviluppatore definisce la logica dei componenti e la loro collocazione nell'interfaccia web, mentre la gestione del Virtual DOM e della comunicazione con il DOM del browser è completamente a carico di React.js

Per l'utilizzatore dell'interfaccia web

- L'impiego del Virtual DOM alleggerisce il processo di rendering dell'interfaccia sul browser («*rendering selettivo*»), con un conseguente aumento delle prestazioni percettibili dall'utilizzatore

Link e riferimenti utili



- Link alla documentazione ufficiale di **ReactJS**:
<https://react.dev/learn>
- Convertitore online da HTML a JSX:
<https://transform.tools/html-to-jsx>



Progettazione di applicazioni web full stack

Sviluppo frontend con React



Prof. Devis Bianchini

Dip. di Università degli Studi di Brescia